AWS Data Lake Engineering — Real-World Scenarios

6 scenarios with architecture diagrams, services, data flow & design decisions

Batch · Streaming · CDC · IoT · Governance (Lake Formation) · Log analytics

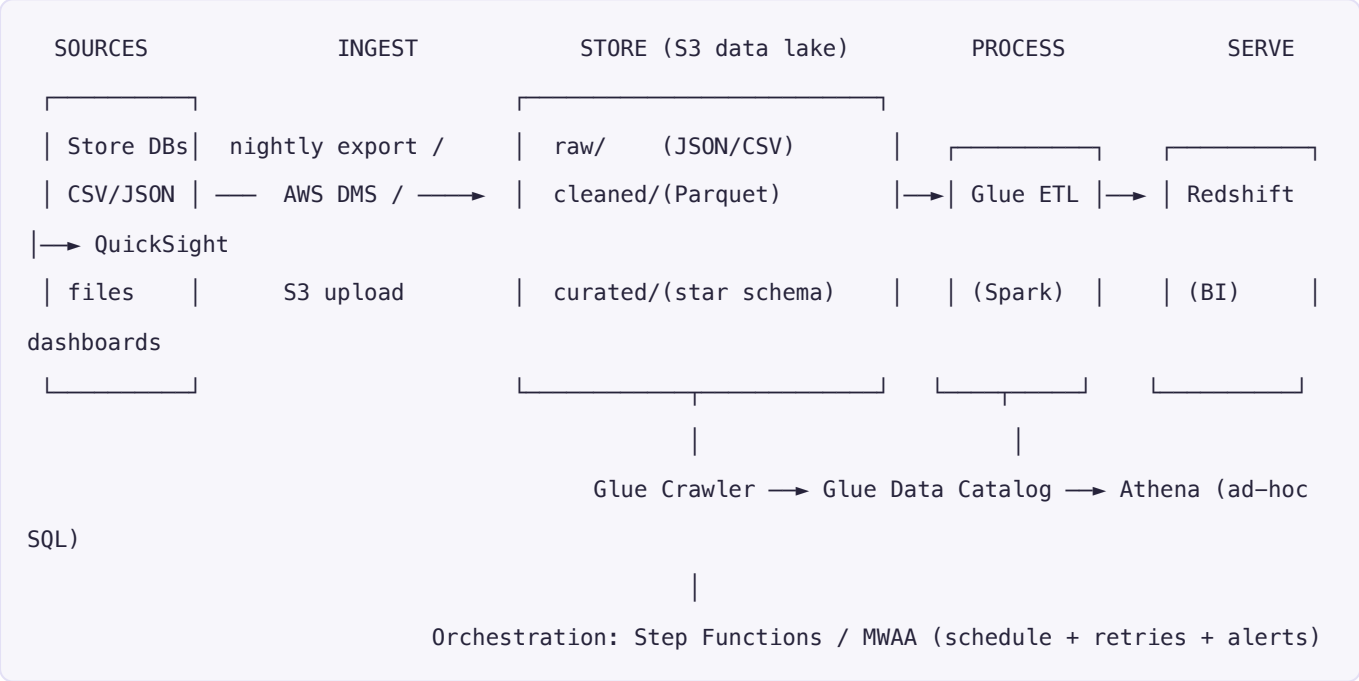
Six realistic scenarios a data engineer builds on AWS. Each has: the **business problem**, an **architecture diagram**, the **services and why**, the **data flow**, the **design decisions**, and the **challenges + how you solve them**. Together they cover the breadth of data lake engineering: batch, streaming, CDC, IoT, governance, and log analytics.

Foundation used in all of them — the "medallion" lake on S3: `raw / bronze` (immutable landing) → `cleaned / silver` (validated, typed) → `curated / gold` (modeled, aggregated). Storage (S3) is decoupled from compute (Glue, Athena, Redshift, EMR), so you scale and pay for each independently.

SCENARIO 1 — Retail batch analytics data lake (the classic)

Business problem: "ShopFast" gets daily order, customer, and product files from many stores. Leadership wants next-morning dashboards: revenue by region, top products, repeat customers.

Architecture



Services & why: S3 (lake storage), **AWS DMS** or scripted upload (ingest), **Glue** (serverless Spark ETL → Parquet, partitioned by date), **Glue Crawler + Data Catalog** (schema), **Athena** (ad-hoc SQL), **Redshift** (fast BI), **QuickSight** (dashboards), **Step Functions/MWAA** (orchestration).

Data flow: files land in `raw/orders/dt=YYYY-MM-DD/` → a **bookmarked** Glue job cleans/dedupes/types them and writes **Parquet** to `cleaned/` → another job models a **star schema** (fact_orders + dim_customer/product/date) into `curated/` and `COPY`s into Redshift → crawler catalogs everything → dashboards query Redshift; analysts use Athena on S3.

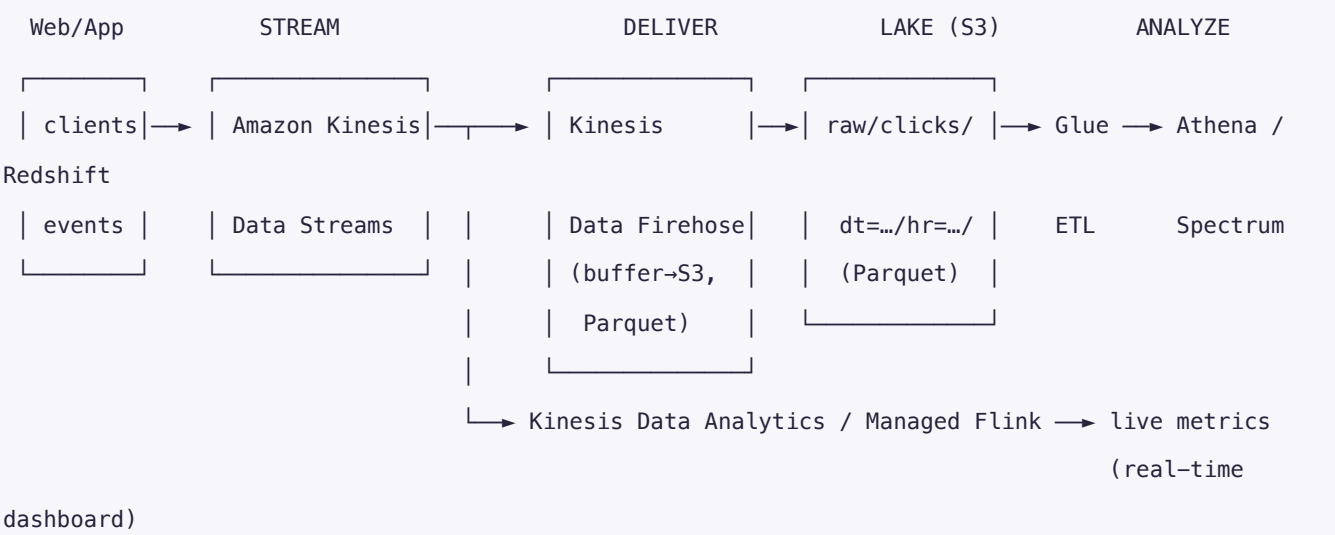
Design decisions: Parquet + date partitioning (scan less), incremental via **job bookmarks**, **idempotent** writes (overwrite the day's partition on rerun), star schema for BI, keep cold history in S3 (Redshift Spectrum) instead of loading years of data.

Challenges → solutions: *small files* → repartition/compact; *late-arriving data* → reprocess the affected date partition; *nightly SLA* → parallelize Glue workers + orchestrate with retries/alerts.

SCENARIO 2 — Real-time clickstream / streaming ingestion

Business problem: A media site wants near-real-time analytics on user clicks (which articles trend *right now*), plus the full history stored cheaply for later analysis.

Architecture



Services & why: Kinesis Data Streams (durable, ordered ingestion of high-velocity events), **Kinesis Data Firehose** (buffers and writes batched **Parquet** to S3 — no code), **Managed Service for Apache Flink** / Kinesis Data Analytics (real-time aggregations on the stream), then the usual **S3** → **Glue** → **Athena/Redshift** for historical analysis.

Data flow: clients send click events to the stream → **two consumers**: (1) Firehose buffers ~1–5 min and lands partitioned Parquet in the lake (the "cold path"); (2) Flink computes rolling counts for a live "trending now" dashboard (the "hot path").

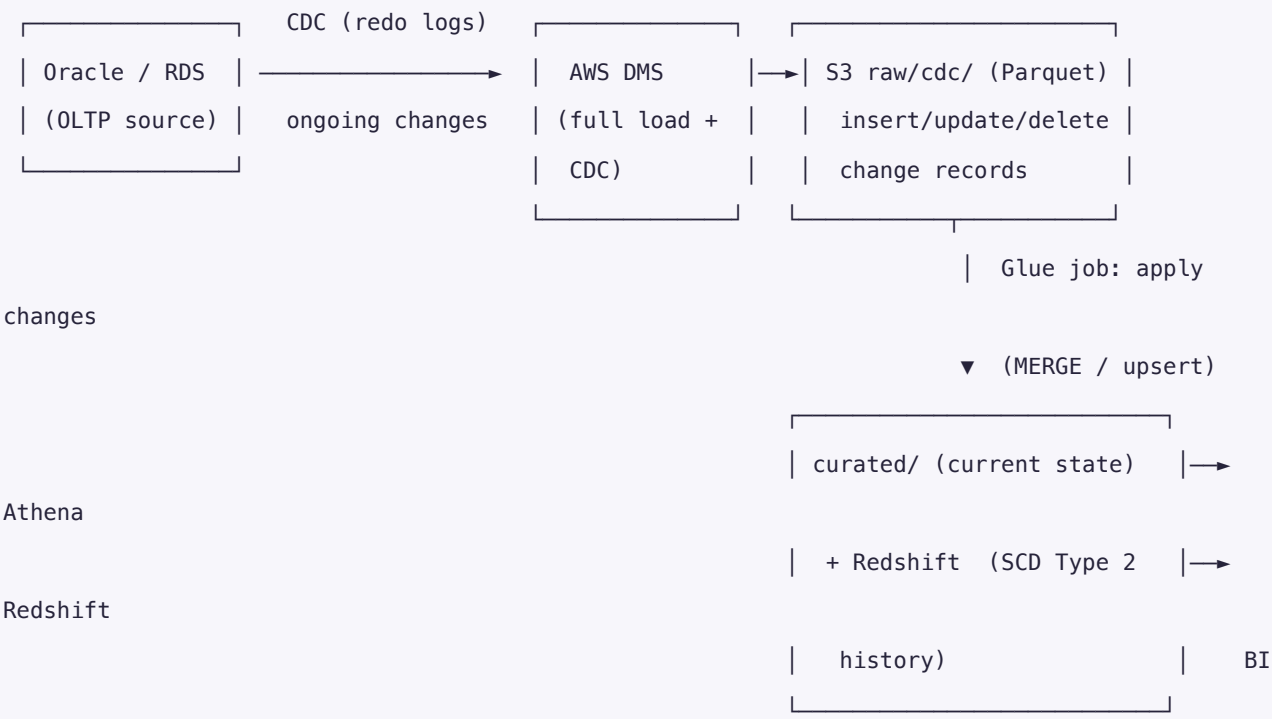
Design decisions: the **hot/cold (Lambda architecture) split** — stream processing for low-latency metrics, batch lake for deep/historical analysis. Firehose does format conversion (JSON→Parquet) and dynamic partitioning so the lake is query-ready.

Challenges → solutions: *ordering/scale* → size Kinesis **shards** to throughput; *tiny-file explosion from streaming* → Firehose buffering + a periodic **compaction** job; *duplicate events* → idempotency keys / dedupe in Flink; *back-pressure* → auto-scaling consumers.

SCENARIO 3 — Change Data Capture (CDC): replicate an operational DB into the lake

Business problem: A bank's core transactions live in an Oracle OLTP database. Analysts need up-to-date data in the lake/warehouse **without hammering the production DB** with queries.

Architecture



Services & why: AWS DMS (does a one-time **full load** then streams ongoing **CDC** from the source's transaction logs — low impact on production), **S3** (lands raw change records), **Glue** (applies inserts/updates/deletes to build current state and history), **Redshift/Athena** (serve).

Data flow: DMS full-loads existing rows, then continuously captures every insert/update/ delete → change records land in S3 → a Glue job **merges** them into a current-state table (upsert) and maintains **SCD Type 2** history (effective/expiry dates + current flag).

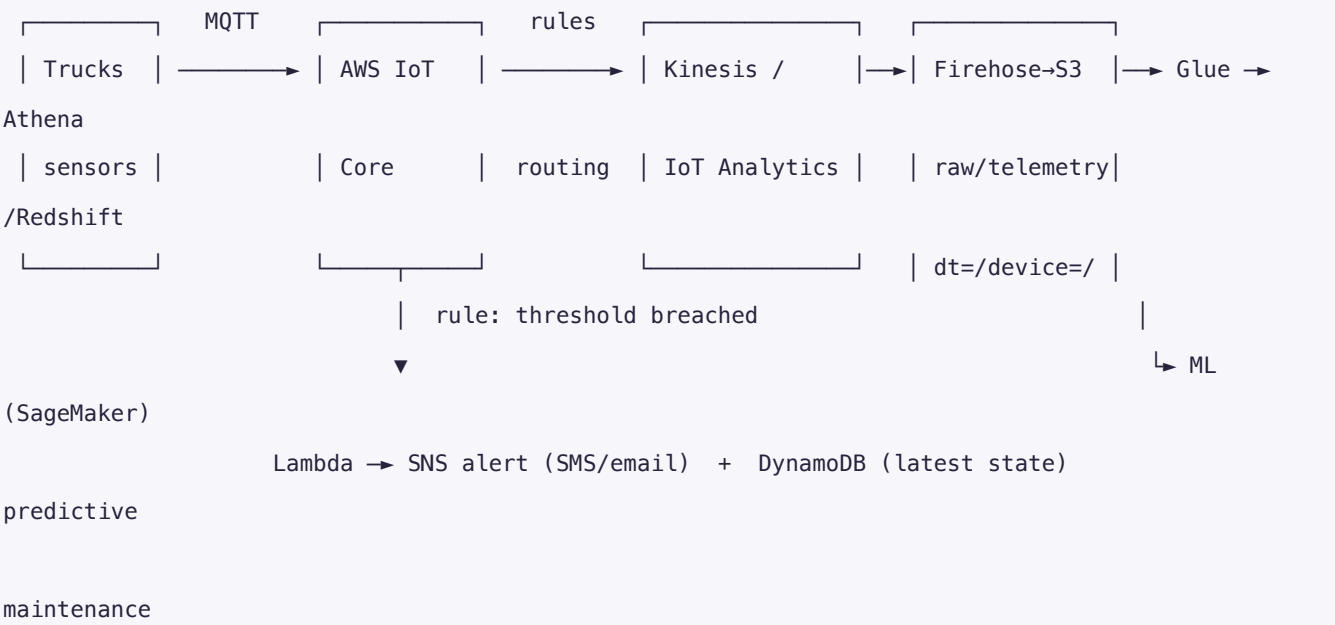
Design decisions: CDC instead of nightly full dumps (fresher data, no source load); staging + **MERGE/upsert** pattern; keep both *current state* (for fast queries) and *full history* (for audit/point-in-time).

Challenges → solutions: *applying deletes/updates in an append-only lake* → merge logic or a table format (**Apache Iceberg/Hudi/Delta**) that supports row-level updates; *schema drift* in the source → schema evolution handling; *ordering of changes* → sequence/commit timestamp.

SCENARIO 4 — IoT / sensor telemetry data lake

Business problem: A logistics fleet streams GPS + engine telemetry from thousands of trucks. The company wants live alerts (engine fault, geofence breach) **and** a lake for long-term maintenance analytics.

Architecture



Services & why: AWS IoT Core (secure MQTT ingestion + **rules engine** to route/ filter), **Kinesis/IoT Analytics** (stream), **Firehose** → **S3** (lake), **Lambda + SNS** (real- time alerts), **DynamoDB** (latest-known state per device for fast lookups), **Glue/Athena/ Redshift** (analytics), **SageMaker** (predictive maintenance ML on the historical lake).

Data flow: devices publish to IoT Core → rules route: (1) threshold breaches trigger a Lambda → **SNS** alert; (2) all telemetry flows to the lake via Firehose (Parquet, partitioned by date/device) for batch analytics and ML training.

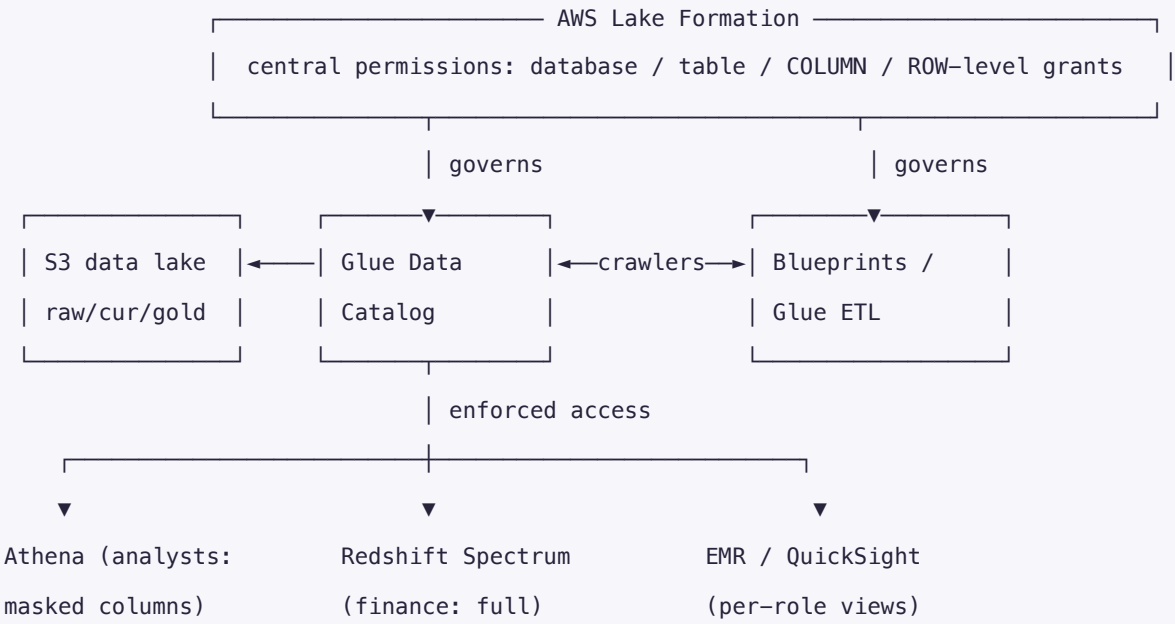
Design decisions: IoT rules filter at the edge of ingestion; **partition by date and device** for efficient queries; DynamoDB for the "current value" hot lookup; the lake feeds **ML** for predictive maintenance.

Challenges → solutions: *huge event volume* → partitioning + Parquet + compaction; *late/out-of-order sensor data* → event-time partitioning + reprocessing; *device schema variety* → a flexible schema + Glue `ResolveChoice`.

SCENARIO 5 — Governed multi-team data lake with Lake Formation

Business problem: A large enterprise has many teams sharing one lake. Finance can see salary columns; analysts cannot. Security and auditors need centralized, fine-grained access control — not per-bucket IAM sprawl.

Architecture



Services & why: AWS Lake Formation (the governance layer — register S3 locations, then grant **database/table/column/row-level** permissions centrally, with **tag-based access control**), on top of **Glue Data Catalog** + **S3**; consumers (**Athena, Redshift Spectrum, EMR, QuickSight**) all enforce the same permissions.

Data flow: data lands and is cataloged as usual → Lake Formation registers the lake and defines who can see which **columns/rows** → every query engine enforces those grants, so analysts querying `employees` automatically get salary columns masked while finance sees them.

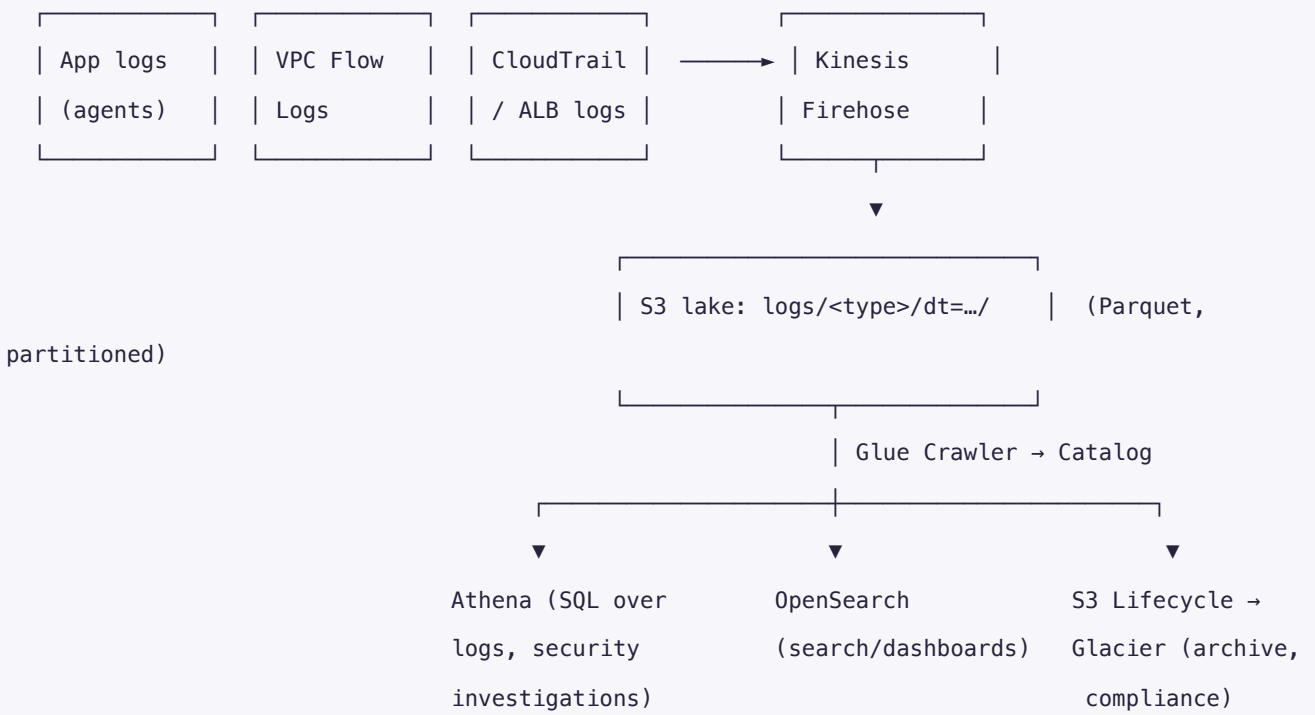
Design decisions: central, fine-grained governance instead of brittle per-bucket IAM; **tag-based** permissions that scale to hundreds of tables; single catalog as the source of truth; **cross-account data sharing** for a data-mesh setup.

Challenges → solutions: *IAM sprawl* → Lake Formation tag-based access; *PII protection* → column masking + row filters; *audit* → CloudTrail + Lake Formation access logs; *multi- account* → LF cross-account grants / data mesh.

SCENARIO 6 — Centralized log & security analytics lake

Business problem: An enterprise wants one place to store and query all logs — application, VPC flow, CloudTrail, ALB — for troubleshooting, security investigations, and compliance.

Architecture



Services & why: log sources → **Kinesis Firehose** (batch to S3 as Parquet) → **S3 lake** (partitioned by type/date) → **Glue Crawler/Catalog** → **Athena** (SQL investigations) and/or **OpenSearch** (full-text search + Kibana dashboards); **S3 Lifecycle** tiers old logs to **Glacier** for cheap long-term compliance retention.

Data flow: all log types funnel through Firehose into a partitioned lake → Athena answers "show me all failed logins from IP X last week" in SQL; OpenSearch powers interactive security dashboards; lifecycle rules archive cold logs automatically.

Design decisions: one lake for all logs (single query surface), Parquet + partitioning (fast, cheap Athena scans), **lifecycle tiering** for cost (hot in S3 Standard, cold in Glacier), dual serving (Athena for SQL, OpenSearch for search).

Challenges → solutions: *massive volume/cost* → partitioning + Parquet + Glacier tiering + scan-limit workgroups; *many schemas* → per-type tables; *real-time alerting* → subscribe a Lambda/OpenSearch alert on the stream.

Cross-cutting principles (true in every scenario)

- **Decouple storage & compute** — S3 is the durable lake; compute (Glue/Athena/Redshift/EMR) is on-demand.
- **Medallion zones** — raw (immutable) → cleaned → curated; always reprocessable from raw.
- **Parquet + partitioning + compression** — scan less = faster + cheaper.
- **Incremental & idempotent** — bookmarks/CDC for new data; reruns don't double-count.

- **Catalog everything** — Glue Data Catalog as the shared schema; govern with Lake Formation.
- **Orchestrate & monitor** — Step Functions/MWAA with retries, alerts, and data-quality gates.
- **Right tool per latency** — batch (Glue/EMR), streaming (Kinesis/Flink), interactive (Athena), high-concurrency BI (Redshift).
- **Cost & security by design** — lifecycle tiering, least-privilege IAM, encryption (SSE-KMS), everything as code (Terraform/CloudFormation).

Quick service cheat (what each is for)

- **S3** — the data lake (storage). **Glue** — serverless Spark ETL + Data Catalog + crawlers.
- **Athena** — serverless SQL on S3. **Redshift** — data warehouse for BI (+ Spectrum → S3).
- **Kinesis (Streams/Firehose/Flink)** — streaming ingest + real-time processing.
- **DMS** — database migration + CDC. **IoT Core** — device ingestion + rules.
- **Lake Formation** — fine-grained governance. **Step Functions / MWAA** — orchestration.
- **OpenSearch** — search/log dashboards. **SageMaker** — ML on the lake.